

TAD Lista, Pila y Cola

Contrato y ejemplos de uso

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Septiembre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Especificar los TAD Lista, Pila y Cola con su dominio, sus operaciones y las precondiciones de cada una.
- Distinguir las tres disciplinas de acceso: por posición, por el último que llegó y por el primero que llegó.
- Escribir funciones que resuelven un problema usando solo el contrato, sin saber cómo está guardada la estructura.
- Ejecutar a mano una secuencia de operaciones y predecir el estado final.
- Enunciar el costo en tiempo y en espacio de cada operación y de cada función escrita contra el contrato.

Objetivos de aprendizaje II

Objetivos de Aprendizaje del curso involucrados

- OA1: diseñar soluciones a problemas pequeños con estructuras de datos simples.
- OA2: escoger la estructura según el costo de las operaciones que el problema necesita.
- OA3: describir ideas con argumentos precisos, aquí en forma de contratos con precondiciones y poscondiciones.

Referencia bibliográfica

R. Thareja, *Data Structures Using C*, 2018, capítulo 2 (tipos abstractos de datos), capítulo 7 (pilas) y capítulo 8 (colas). N. Kalicharan, *Data Structures in C*, 2008, capítulo 4 (pilas y colas).

Contenido I

- 1 Lo que dejó el TAD Racional
- 2 Tres maneras de guardar muchos
- 3 TAD Lista
- 4 TAD Pila
- 5 TAD Cola
- 6 Los tres contratos, juntos

Contenido I

- 1 Lo que dejó el TAD Racional
- 2 Tres maneras de guardar muchos
- 3 TAD Lista
- 4 TAD Pila
- 5 TAD Cola
- 6 Los tres contratos, juntos

Un tipo se describe por lo que hace, no por lo que guarda l

El contrato del TAD Racional

- Un conjunto de valores: $\{ p/q : q \neq 0 \}$.
- Unas operaciones con su firma: `crear`, `numerador`, `denominador`.
- Unas ecuaciones y una precondition: `crear(p, q)` exige $q \neq 0$.

Lo que el contrato compró

- `sumar`, `multiplicar` e `igual` se escribieron una sola vez y sirvieron para las dos representaciones.
- La precondition se vigila con `assert`: el error se detiene donde nace, no diez llamadas después.
- El puntero `this` marca de qué lado de la barrera está cada línea.

Un tipo se describe por lo que hace, no por lo que guarda II

Hoy el mismo esquema, tres veces, para tipos que guardan **muchos** valores en vez de uno.

Contenido I

- 1 Lo que dejó el TAD Racional
- 2 Tres maneras de guardar muchos
- 3 TAD Lista
- 4 TAD Pila
- 5 TAD Cola
- 6 Los tres contratos, juntos

¿Qué tienen en común? I

- El historial de un navegador y el botón *atrás*.
- La fila para pagar en la cafetería.
- Una lista de reproducción donde se puede saltar a la canción 7.

Lo común

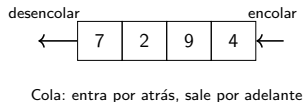
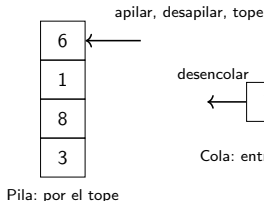
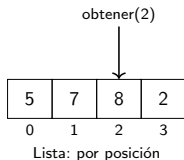
Los tres guardan una colección **lineal**: cada elemento tiene un anterior y un siguiente, salvo los extremos.

Lo distinto

Por dónde se entra y por dónde se sale.

- En la lista de reproducción, por **cualquier posición**.
- En el historial, por **el último que llegó**.
- En la fila, por **el primero que llegó**.

Las tres disciplinas, en un dibujo I



- Los tres son secuencias. El contrato de cada uno decide qué posiciones se pueden tocar.
- Menos operaciones no es menos poder: es una promesa más fuerte sobre el orden, y la implementación la cobra más barata.

Definición I

Definición (Estructura lineal)

Una colección finita de elementos del mismo tipo en la que existe un orden: hay un primero, hay un último y cada elemento, salvo el último, tiene un único siguiente.

La notación de hoy

Una secuencia se escribe entre paréntesis angulares:

$$\langle e_0, e_1, \dots, e_{n-1} \rangle, \quad n \geq 0.$$

La secuencia vacía es $\langle \rangle$, y n es su tamaño.

- Thareja, capítulo 2: el TAD dice *qué* hace cada operación; la representación dice *cómo*.

Definición II

- La notación no dice si los elementos están en un arreglo o enlazados por punteros. Eso es tema de las sesiones de implementación.

Contenido I

- 1 Lo que dejó el TAD Racional
- 2 Tres maneras de guardar muchos
- 3 TAD Lista**
- 4 TAD Pila
- 5 TAD Cola
- 6 Los tres contratos, juntos

La especificación del TAD Lista I

Dominio

$$\{ \langle e_0, \dots, e_{n-1} \rangle : n \geq 0, e_i \in \text{Elemento} \}$$

La especificación del TAD Lista II

Interfaz: firmas, con $n = \text{tamano}(l)$

crear : $\rightarrow \text{Lista}$

insertar : $\text{Lista} \times \text{Entero} \times \text{Elemento} \rightarrow \text{Lista}$ exige $0 \leq p \leq n$

eliminar : $\text{Lista} \times \text{Entero} \rightarrow \text{Lista}$ exige $0 \leq p < n$

obtener : $\text{Lista} \times \text{Entero} \rightarrow \text{Elemento}$ exige $0 \leq p < n$

asignar : $\text{Lista} \times \text{Entero} \times \text{Elemento} \rightarrow \text{Lista}$ exige $0 \leq p < n$

agregar : $\text{Lista} \times \text{Elemento} \rightarrow \text{Lista}$

tamano : $\text{Lista} \rightarrow \text{Entero}$

vacía : $\text{Lista} \rightarrow \text{Booleano}$

La especificación del TAD Lista III

Tres familias de operaciones

- **Constructoras:** producen una lista desde nada. crear.
 - **Modificadoras:** reciben una lista y la cambian. insertar, eliminar, asignar, agregar.
 - **Analizadoras:** preguntan sin cambiar nada. obtener, tamaño, vacía.
-
- Con el TAD Racional bastaban un constructor y dos observadores porque el valor no cambiaba. Una lista sí cambia: por eso aparecen las modificadoras.
 - Las precondiciones hablan de posiciones. Fíjese en la única que admite $p = n$.

Precondición y poscondición de las modificadoras I

Sea $l = \langle e_0, \dots, e_{n-1} \rangle$ antes de la operación.

insertar(l, p, e)

Exige: $0 \leq p \leq n$.

Deja: $l = \langle e_0, \dots, e_{p-1}, e, e_p, \dots, e_{n-1} \rangle$, de tamaño $n + 1$.

eliminar(l, p)

Exige: $0 \leq p < n$.

Deja: $l = \langle e_0, \dots, e_{p-1}, e_{p+1}, \dots, e_{n-1} \rangle$, de tamaño $n - 1$.

asignar(l, p, v) y **agregar**(l, e)

Exige: $0 \leq p < n$ para asignar; nada para agregar.

Deja: $e_p = v$ y lo demás igual; o bien $l = \langle e_0, \dots, e_{n-1}, e \rangle$.

Precondición y poscondición de las modificadoras II

Pregunta

Con $l = \langle 5, 8 \rangle$, ¿qué deja insertar($l, 1, 7$)? ¿Y insertar($l, 2, 7$)?
¿Y insertar($l, 3, 7$)?

- $\langle 5, 7, 8 \rangle$: el 7 queda en la posición 1 y el 8 se corre.
- $\langle 5, 8, 7 \rangle$: $p = n$ es agregar al final.
- Nada: $p = 3 > n$ viola la precondición. El contrato no dice qué pasa, y eso es deliberado: quien llama responde por p .

La caja del TAD Lista I

TAD Lista

$\{ \langle e_0, \dots, e_{n-1} \rangle : n \geq 0, e_i \in \text{Elemento} \}$

crear : $\rightarrow$ Lista $O(1)$

insertar : Lista $\times$ Entero $\times$ Elemento $\rightarrow$ Lista, exige $0 \leq p \leq n$ $O(n)$

eliminar : Lista $\times$ Entero $\rightarrow$ Lista, exige $0 \leq p < n$ $O(n)$

obtener : Lista $\times$ Entero $\rightarrow$ Elemento, exige $0 \leq p < n$ $O(1)$

asignar : Lista $\times$ Entero $\times$ Elemento $\rightarrow$ Lista, exige $0 \leq p < n$ $O(1)$

agregar : Lista $\times$ Elemento $\rightarrow$ Lista $O(1)$

tamano : Lista $\rightarrow$ Entero vacia : Lista $\rightarrow$ Booleano $O(1)$

Espacio $\rightarrow O(\text{espacio})$

Todo lo que el resto del programa puede saber de una lista está en esta caja. Ni arreglos ni punteros: eso vive debajo de la barrera.

El contrato en C++: la parte pública de la clase I

```
typedef int Elemento;

class Lista {
private:
    // la representacion llega en las sesiones de implementacion
public:
    Lista(); // crear
    void insertar(int p, Elemento e); // 0 <= p <= tamano()
    void eliminar(int p); // 0 <= p < tamano()
    Elemento obtener(int p); // 0 <= p < tamano()
    void asignar(int p, Elemento e); // 0 <= p < tamano()
    void agregar(Elemento e);
    int tamano();
    bool vacia();
};
```

El contrato en C++: la parte pública de la clase II

- La lista sobre la que actúa cada operación viaja en `this`; por eso la firma en C++ tiene un argumento menos que la de la caja.
- Hoy trabajamos con una implementación provisional escondida en `lista.h`. El programa de esta sesión no la ve, y por eso sobrevivirá cuando la cambiemos.

Las operaciones, una por una I

```
int main() {  
    Lista l;  
    imprimir(l);  
    l.agregar(5);  
    l.agregar(8);  
    imprimir(l);  
    l.insertar(1, 7);  
    imprimir(l);  
    l.eliminar(0);  
    imprimir(l);  
    l.asignar(1, 2);  
    imprimir(l);  
    printf("tamano: %d, vacia: %d\n", l.tamano(), l.vacia());  
    return 0;  
}
```

Las operaciones, una por una II

Traza: el estado después de cada línea

Operación	Estado	Salida
Lista 1	$\langle \rangle$	$< >$
agregar(5), agregar(8)	$\langle 5, 8 \rangle$	$< 5 \ 8 >$
insertar(1, 7)	$\langle 5, 7, 8 \rangle$	$< 5 \ 7 \ 8 >$
eliminar(0)	$\langle 7, 8 \rangle$	$< 7 \ 8 >$
asignar(1, 2)	$\langle 7, 2 \rangle$	$< 7 \ 2 >$
tamano(), vacia()	$\langle 7, 2 \rangle$	2, 0

- Después de eliminar(0) el 8 ya no está en la posición 2 sino en la 1: eliminar corre todo lo que sigue.
- Cada fila es una poscondición cumplida. Verificarlo a mano es la forma de leer un contrato.

Lo que cuesta cada operación de la lista I

Con la implementación que corre hoy, un arreglo de CAPACIDAD casillas y un contador n :

Operación	Tiempo	Por qué
crear	$O(1)$	poner n en cero
obtener(p), asignar(p, e)	$O(1)$	se calcula la dirección de la casilla
agregar(e)	$O(1)$	se escribe en la casilla n
insertar(p, e)	$O(n - p)$, peor caso $O(n)$	se corren las casillas de p a $n - 1$
eliminar(p)	$O(n - p)$, peor caso $O(n)$	se corren las casillas de $p + 1$ a $n - 1$
tamano, vacia	$O(1)$	se lee n

Espacio

$\Theta(n)$ para n elementos. Hoy se reservan las CAPACIDAD casillas de antemano, así que la estructura ocupa $\Theta(\text{CAPACIDAD})$ aunque esté vacía; una implementación que crezca según haga falta cobra $\Theta(n)$.

Lo que cuesta cada operación de la lista II

- Insertar o eliminar al **final** cuesta $O(1)$; al **frente**, $O(n)$. Esa asimetría es la que van a explotar la pila y la cola.
- Las implementaciones que construiremos más adelante tienen que sostener estos costos o mejorarlos, y ahí se demuestran.

El área del programador: recorrer l

```
void imprimir(Lista &l) {  
    int p = 0;  
    printf("<");  
    while (p < l.tamano()) {  
        printf(" %d", l.obtener(p));  
        p = p + 1;  
    }  
    printf(" >\n");  
}
```

- La lista llega por referencia: copiarla costaría tanto como recorrerla, y la función no la modifica.
- Solo aparecen tamaño y obtener. Esta función no sabe si hay un arreglo debajo, y no lo necesita.
- El patrón *posición de 0 a tamaño* es el recorrido de toda lista; con pilas y colas no va a existir, porque no hay posiciones.

El área del programador: mezclar dos listas ordenadas I

Problema

a y b vienen ordenadas de menor a mayor. Construir una lista nueva con todos los elementos de ambas, ordenada, sin ordenar nada.

Idea

Dos posiciones, i sobre a y j sobre b . En cada paso se copia el menor de los dos frentes y avanza solo esa posición. Cuando una lista se agota, se copia el resto de la otra.

- Es el mismo paso de *combinar* que el algoritmo de ordenamiento por mezcla usa sobre arreglos. Aquí está escrito contra el contrato de Lista.

El área del programador: mezclar dos listas ordenadas II

```
// copia al final de r los elementos de l desde la posicion p  
void copiarDesde(Lista &r, Lista &l, int p) {  
    while (p < l.tamano()) {  
        r.agregar(l.obtener(p));  
        p = p + 1;  
    }  
}
```

- El “resto” tiene nombre propio: es la misma función para a y para b , y mezclar queda con una sola idea.

El área del programador: mezclar dos listas ordenadas III

```
Lista mezclar(Lista &a, Lista &b) {  
    Lista r;  
    int i = 0;  
    int j = 0;  
    while (i < a.tamano() && j < b.tamano()) {  
        if (a.obtener(i) <= b.obtener(j)) {  
            r.agregar(a.obtener(i));  
            i = i + 1;  
        } else {  
            r.agregar(b.obtener(j));  
            j = j + 1;  
        }  
    }  
    copiarDesde(r, a, i);  
    copiarDesde(r, b, j);  
    return r;  
}
```

La traza de mezclar con $\langle 4, 5, 9 \rangle$ y $\langle 1, 5, 8 \rangle$ I

i	j	comparación	se copia	r
0	0	$4 \leq 1$ no	$b_0 = 1$	$\langle 1 \rangle$
0	1	$4 \leq 5$ sí	$a_0 = 4$	$\langle 1, 4 \rangle$
1	1	$5 \leq 5$ sí	$a_1 = 5$	$\langle 1, 4, 5 \rangle$
2	1	$9 \leq 5$ no	$b_1 = 5$	$\langle 1, 4, 5, 5 \rangle$
2	2	$9 \leq 8$ no	$b_2 = 8$	$\langle 1, 4, 5, 5, 8 \rangle$
2	3	b agotada	copiarDesde: 9	$\langle 1, 4, 5, 5, 8, 9 \rangle$

- El $\leq$ en vez de $<$ decide quién va primero en un empate: el de a . Con $<$ el resultado sería el mismo aquí, pero no si los elementos fueran registros con más campos.
- Cada elemento se copia una sola vez: el costo es el tamaño de a más el de b , contando llamadas al contrato.
- Salida real del programa: $\langle 1\ 4\ 5\ 5\ 8\ 9 \rangle$.

La traza de mezclar con $\langle 4, 5, 9 \rangle$ y $\langle 1, 5, 8 \rangle$ II

Costos, con $n = \text{tamano}(a) + \text{tamano}(b)$

- imprimir: $\Theta(n)$ en tiempo, n llamadas a obtener de $O(1)$ cada una; $\Theta(1)$ de espacio aparte de la lista, solo la posición p .
- mezclar: $\Theta(n)$ en tiempo, porque cada vuelta de cualquiera de los tres ciclos copia un elemento y avanza una posición, y cada agregar cuesta $O(1)$; $\Theta(n)$ de espacio, la lista nueva que devuelve.
- Si mezclar se escribiera con insertar en una posición intermedia en vez de agregar al final, cada copia costaría $O(n)$ y el total subiría a $O(n^2)$. El contrato es el mismo; el costo lo decide qué operación se elige.

Ejercicio en parejas: revertir l

Enunciado

Escriba `revertir(l)`, que devuelve una lista nueva con los mismos elementos de `l` en orden inverso, usando solo el contrato. Para $\langle 1, 4, 5, 5, 8, 9 \rangle$ debe dar $\langle 9, 8, 5, 5, 4, 1 \rangle$.

Pistas

- Hay dos caminos: recorrer `l` de atrás hacia adelante y agregar, o recorrer de adelante hacia atrás y usar `insertar` en una posición fija.
- Antes de escribir, diga qué precondition cumple cada llamada que hace.

Ejercicio en parejas: revertir II

Una solución: insertar siempre al frente

```
Lista revertir(Lista &l) {  
    Lista r;  
    int p = 0;  
    while (p < l.tamano()) {  
        r.insertar(0, l.obtener(p));  
        p = p + 1;  
    }  
    return r;  
}
```

- `insertar(r, 0, .)` cumple su precondition siempre, porque $0 \leq 0 \leq \text{tamano}(r)$ vale incluso con r vacía.
- El otro camino, con p desde $\text{tamano}(l) - 1$ hasta 0 y agregar, da la misma lista.

Ejercicio en parejas: revertir III

Las dos versiones cuestan distinto

- Con `insertar(0, ·)`: la k -ésima inserción corre los $k - 1$ elementos que ya están, así que el total es $0 + 1 + \dots + (n - 1) = n(n - 1)/2$: $\Theta(n^2)$ en tiempo.
 - Con agregar desde el final: n operaciones de $O(1)$, $\Theta(n)$ en tiempo.
 - Las dos usan $\Theta(n)$ de espacio: la lista nueva.
-
- Mismo contrato, misma salida, y una versión es cuadrática donde la otra es lineal. Antes de escribir una función contra un contrato hay que mirar la tabla de costos de sus operaciones.

Contenido I

- 1 Lo que dejó el TAD Racional
- 2 Tres maneras de guardar muchos
- 3 TAD Lista
- 4 TAD Pila**
- 5 TAD Cola
- 6 Los tres contratos, juntos

La pila: el último que entra es el primero que sale

Idea

Una lista en la que solo se toca un extremo, el **tope**. Se agrega por el tope, se quita por el tope y solo se puede ver el tope. Thareja, capítulo 7: disciplina LIFO, *last in, first out*.

Dónde aparece

- Deshacer y rehacer en un editor.
- El botón *atrás* del navegador.
- Las llamadas a funciones: la que llamó espera a que termine la última que entró.

Pregunta

Si solo se puede ver el tope, ¿cómo se imprime una pila completa? Y después de imprimirla, ¿qué queda de ella?

La caja del TAD Pila I

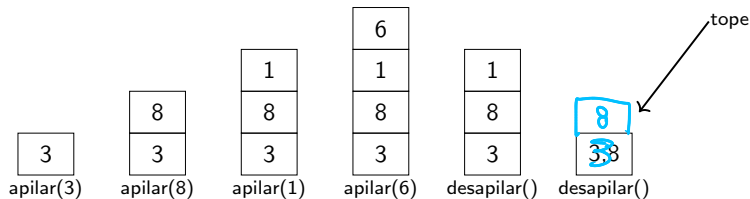
TAD Pila
$\{ \langle e_0, \dots, e_{n-1} \rangle : n \geq 0 \},$ con e_{n-1} el tope
crear : $\rightarrow$ Pila apilar : $\text{Pila} \times \text{Elemento} \rightarrow \text{Pila}$ desapilar : $\text{Pila} \rightarrow \text{Pila}$, exige $n > 0$ tope : $\text{Pila} \rightarrow \text{Elemento}$, exige $n > 0$ tamano : $\text{Pila} \rightarrow \text{Entero}$ vacia : $\text{Pila} \rightarrow \text{Booleano}$

Poscondiciones, con $p = \langle e_0, \dots, e_{n-1} \rangle$

apilar(p, e) deja $\langle e_0, \dots, e_{n-1}, e \rangle$. desapilar(p) deja $\langle e_0, \dots, e_{n-2} \rangle$. tope(p) = e_{n-1} .

- Ninguna operación recibe una posición. Esa es toda la diferencia con la lista.

Apilar y desapilar, paso a paso I



- Después de cuatro apilar, `tope()` es 6; el 3 está abajo de todo y no hay manera de leerlo sin desapilar tres veces.
- Cada `desapilar` descarta el tope. El valor que se quita no se devuelve: quien lo quiera, lo lee antes con `tope`.

Lo que cuesta cada operación de la pila I

Operación	Tiempo	Por qué
crear	$O(1)$	poner n en cero
apilar(e)	$O(1)$	se escribe en la casilla n
desapilar	$O(1)$	se descuenta n ; nada se corre
tope	$O(1)$	se lee la casilla $n - 1$
tamano, vacia	$O(1)$	se lee n

Espacio

$\Theta(n)$ para n elementos, con la misma salvedad de la lista: hoy, CAPACIDAD casillas reservadas de antemano.

- Todas constantes, porque la pila solo toca el extremo donde la lista era barata. Es lo que se gana al prometer menos: un contrato sin posiciones no necesita correr nada.

El contrato en C++ y la precondition vigilada I

```
class Pila {  
private:  
    // la representacion llega en las sesiones de implementacion  
public:  
    Pila();                // crear  
    void apilar(Elemento e);  
    void desapilar();      // exige !vacía()  
    Elemento tope();       // exige !vacía()  
    int tamaño();  
    bool vacía();  
};
```

El contrato en C++ y la precondition vigilada II

```
int main() {  
    Pila p;  
    p.apilar(4);  
    printf("tope: %d\n", p.tope());  
    p.desapilar();  
    printf("vacía: %d\n", p.vacia());  
    p.desapilar();  
    printf("esta línea no se alcanza\n");  
    return 0;  
}
```

El contrato en C++ y la precondition vigilada III

Salida real

```
tope: 4  
vacía: 1  
pila_precondicion: pila.h:24: void Pila::desapilar():  
  Assertion `n > 0' failed.
```

- La segunda llamada a desapilar viola $n > 0$ y el assert de la implementación detiene el programa ahí. Sin él, habría un tamaño negativo y el error aparecería mucho después, lejos de su causa.

Imprimir una pila: primer intento I

```
void imprimirV1(Pila &p) {  
    while (!p.vacia()) {  
        printf(" %d", p.tope());  
        p.desapilar();  
    }  
    printf("\n");  
}
```

Con $\langle 3, 8, 1, 6 \rangle$

Imprime 6 1 8 3, del tope hacia abajo. Y después, tamaño() vale 0.

Imprimir una pila: primer intento II

El precio de mirar

Con el contrato de Pila no hay forma de ver el segundo elemento sin quitar el primero. Una función que “solo imprime” y recibe la pila por referencia la deja vacía. Si el llamador la necesitaba después, la perdió.

Imprimir una pila y dejarla como estaba l

```
void imprimir(Pila &p) {  
    Pila aux;  
    while (!p.vacia()) {  
        printf(" %d", p.tope());  
        aux.apilar(p.tope());  
        p.desapilar();  
    }  
    while (!aux.vacia()) {  
        p.apilar(aux.tope());  
        aux.desapilar();  
    }  
    printf("\n");  
}
```

Imprimir una pila y dejarla como estaba II

Traza con $p = \langle 3, 8, 1, 6 \rangle$

Paso	p	aux	Salida
inicio	$\langle 3, 8, 1, 6 \rangle$	$\langle \rangle$	
primer ciclo, 4 vueltas	$\langle \rangle$	$\langle 6, 1, 8, 3 \rangle$	6 1 8 3
segundo ciclo, 4 vueltas	$\langle 3, 8, 1, 6 \rangle$	$\langle \rangle$	

- Al pasar todo a aux, el orden se invierte: el 3 que estaba abajo queda en el tope de aux. Al devolverlo se invierte otra vez y p queda igual.
- Dos pasadas por la pila: el doble de trabajo que la versión 1, a cambio de no destruir el argumento. Es el precio de un contrato que solo deja mirar el tope.

Imprimir una pila y dejarla como estaba III

Costos, con $n = \text{tamano}(p)$

- Versión 1: $\Theta(n)$ en tiempo (n tope y n desapilar, de $O(1)$ cada uno) y $\Theta(1)$ de espacio aparte de la pila; pero la deja vacía.
- Versión 2: $\Theta(n)$ en tiempo, $2n$ operaciones de $O(1)$, y $\Theta(n)$ de espacio: la pila auxiliar llega a tener los n elementos.

Ejercicio en parejas: eliminar debajo del tope I

Enunciado

Escriba `eliminarDesdeTope( $p, k$ )`, que quita el elemento que está k lugares debajo del tope ($k = 0$ es el tope) y deja el resto en su orden. Con $p = \langle 3, 8, 1, 6 \rangle$ y $k = 2$ debe quedar $\langle 3, 1, 6 \rangle$.

Pistas

- Escriba primero la precondition sobre k .
- El truco de la pila auxiliar sirve otra vez: ¿cuántos elementos hay que mover para llegar al que sobra?

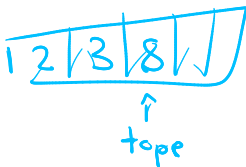
Ejercicio en parejas: eliminar debajo del tope II

Una solución

```
// exige  $0 \leq k < \text{tamano}()$ 
void eliminarDesdeTope(Pila &p, int k) {
    Pila aux;
    int i = 0;
    while (i < k) {
        aux.apilar(p.tope());
        p.desapilar();
        i = i + 1;
    }
    p.desapilar();
    while (!aux.vacia()) {
        p.apilar(aux.tope());
        aux.desapilar();
    }
}
```

Ejercicio en parejas: eliminar debajo del tope III

- Se mueven k elementos, se descarta uno y se devuelven los k .
Salida real tras la llamada: 6 1 3.
- La precondition $k < \text{tamano}()$ es lo que hace legal el desapilar suelto de la mitad.
- Costo: $\Theta(k)$ en tiempo ($2k + 1$ operaciones de $O(1)$) y $\Theta(k)$ de espacio para la auxiliar. No depende de n : lo que está debajo del elemento eliminado no se toca.



Contenido I

- 1 Lo que dejó el TAD Racional
- 2 Tres maneras de guardar muchos
- 3 TAD Lista
- 4 TAD Pila
- 5 TAD Cola**
- 6 Los tres contratos, juntos

La cola: el primero que entra es el primero que sale I

Idea

Una lista en la que se entra por un extremo, el **final**, y se sale por el otro, el **frente**. Thareja, capítulo 8: disciplina FIFO, *first in, first out*.

Dónde aparece

- La fila de atención: turnos, cajas, impresoras compartidas.
- Las tareas que un sistema atiende en el orden en que llegan.
- El recorrido por niveles de un árbol, que veremos más adelante.

La cola: el primero que entra es el primero que sale II

Pregunta

En la pila, imprimir sin destruir costó una pila auxiliar. En la cola, ¿hace falta? La respuesta está en el recorrido que usa el ejemplo que sigue.

La caja del TAD Cola I

TAD Cola
$\{ \langle e_0, \dots, e_{n-1} \rangle : n \geq 0 \}$, con e_0 el frente
crear $\rightarrow$ Cola encolar $: \text{Cola} \times \text{Elemento} \rightarrow \text{Cola}$ desencolar $: \text{Cola} \rightarrow \text{Cola}$, exige $n > 0$ frente $: \text{Cola} \rightarrow \text{Elemento}$, exige $n > 0$ tamano $: \text{Cola} \rightarrow \text{Entero}$ vacia $: \text{Cola} \rightarrow \text{Booleano}$

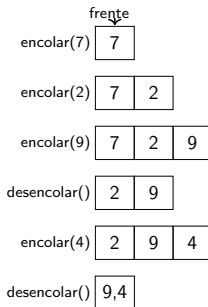
Poscondiciones, con $c = \langle e_0, \dots, e_{n-1} \rangle$

encolar(c, e) deja $\langle e_0, \dots, e_{n-1}, e \rangle$. desencolar(c) deja $\langle e_1, \dots, e_{n-1} \rangle$. frente(c) = e_0 .

La caja del TAD Cola II

- Compare con la pila: encolar y apilar dejan lo mismo. La diferencia está en cuál extremo quita desencolar.

Encolar y desencolar, paso a paso I



- Lo que entra se pone al final; lo que sale, sale del frente. El 7 fue el primero en entrar y el primero en salir.
- En C++ la clase se declara igual que la pila, cambiando `apilar`, `desapilar` y `tope` por `encolar`, `desencolar` y `frente`, con las mismas dos precondiciones.

Lo que cuesta cada operación de la cola I

Con la implementación que corre hoy, un arreglo circular: se guarda la posición del frente y el contador n , y quitar el frente es avanzar esa posición.

Operación	Tiempo	Por qué
crear	$O(1)$	poner el frente y n en cero
encolar(e)	$O(1)$	se escribe en la casilla del final
desencolar	$O(1)$	se avanza la posición del frente; nada se corre
frente	$O(1)$	se lee la casilla del frente
tamano, vacia	$O(1)$	se lee n

Espacio

$\Theta(n)$ para n elementos; hoy CAPACIDAD casillas reservadas, como en la lista y la pila.

Lo que cuesta cada operación de la cola II

- Si desencolar corriera todo el arreglo una casilla, costaría $O(n)$ y un recorrido que da la vuelta costaría $O(n^2)$. El arreglo circular es lo que deja la cola en costos constantes; cómo se construye es tema de la sesión de implementación de la cola.

El área del programador: particionar una cola I

Problema

Dada una cola c y un valor v , reordenarla de modo que queden primero los elementos menores que v y después los demás, cada grupo en el orden en que estaba. Con $\langle 7, 2, 9, 4, 5 \rangle$ y $v = 5$ debe quedar $\langle 2, 4, 7, 9, 5 \rangle$.

Idea

Recorrer la cola una vuelta completa: cada elemento sale por el frente y vuelve a entrar por el final de c si es menor que v , o se guarda en una cola auxiliar si no. Al terminar, la auxiliar se vacía al final de c .

- El recorrido de una cola es *dar la vuelta*: sacar del frente y volver a encolar, tantas veces como elementos había. Por eso hay que leer *tamaño* antes de empezar.

El área del programador: particionar una cola II

```
void particionar(Cola &c, Elemento v) {  
    Cola mayores;  
    int i = 0;  
    int n = c.tamano();  
    while (i < n) {  
        Elemento x = c.frente();  
        c.desencolar();  
        if (x < v) {  
            c.encolar(x);  
        } else {  
            mayores.encolar(x);  
        }  
        i = i + 1;  
    }  
    while (!mayores.vacia()) {  
        c.encolar(mayores.frente());  
        mayores.desencolar();  
    }  
}
```

La traza de particionar con $\langle 7, 2, 9, 4, 5 \rangle$ y $v = 5$ |

i	x y decisión	c después	mayores
0	$7 < 5$ no	$\langle 2, 9, 4, 5 \rangle$	$\langle 7 \rangle$
1	$2 < 5$ sí	$\langle 9, 4, 5, 2 \rangle$	$\langle 7 \rangle$
2	$9 < 5$ no	$\langle 4, 5, 2 \rangle$	$\langle 7, 9 \rangle$
3	$4 < 5$ sí	$\langle 5, 2, 4 \rangle$	$\langle 7, 9 \rangle$
4	$5 < 5$ no	$\langle 2, 4 \rangle$	$\langle 7, 9, 5 \rangle$
fin	vaciar mayores	$\langle 2, 4, 7, 9, 5 \rangle$	$\langle \rangle$

- Si el ciclo usara `!c.vacia()` en vez de $i < n$, los menores que vuelven a entrar se procesarían otra vez y el ciclo no terminaría. El tamaño se lee una vez, antes.
- Cada grupo conserva su orden porque la cola es FIFO: es la poscondición de `encolar` aplicada n veces.
- Salida real: 2 4 7 9 5, y `tamano()` sigue en 5.

La traza de particionar con $\langle 7, 2, 9, 4, 5 \rangle$ y $v = 5$ II

- Costo: $\Theta(n)$ en tiempo, una vuelta completa más el vaciado de mayores, todo con operaciones de $O(1)$; y $O(n)$ de espacio para la auxiliar, que en el peor caso (todos $\geq v$) recibe los n elementos.

Contenido I

- 1 Lo que dejó el TAD Racional
- 2 Tres maneras de guardar muchos
- 3 TAD Lista
- 4 TAD Pila
- 5 TAD Cola
- 6 Los tres contratos, juntos

Lo que cada contrato permite I

	Lista	Pila	Cola
entra por	cualquier p	el tope	el final
sale por	cualquier p	el tope	el frente
se lee	cualquier p	solo el tope	solo el frente
recorrer	p de 0 a $n - 1$	vaciarse y rehacer	dar la vuelta
precondición típica	$0 \leq p < n$	$n > 0$	$n > 0$
costo de entrar y salir	$O(1)$ al final, $O(n)$ en medio	$O(1)$	$O(1)$
espacio	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

- Todo lo que hace una pila o una cola se puede hacer con una lista. Se escogen igual porque prometen menos: quien las lee sabe de inmediato en qué orden salen las cosas, y quien las implementa puede cobrar cada operación a costo constante.
- Los costos de la tabla son los de la implementación que corre hoy. Las que construiremos más adelante tienen que sostenerlos, y ahí cada fila se demuestra.

Los mismos contratos con los nombres de la STL I

Hoy	En la STL de C++	Contenedor
agregar, insertar, eliminar	push_back, insert, erase	vector, list
obtener, asignar	at, operator[]	vector
apilar, desapilar, tope	push, pop, top	stack
encolar, desencolar, frente	push, pop, front	queue
tamano, vacia	size, empty	todos

- La biblioteca estándar trae estos tres contratos ya implementados. Las funciones de hoy se pueden reescribir sobre ellos cambiando solo los nombres, y eso es lo que sigue en el curso.
- pop en la STL tampoco devuelve el valor. La decisión de contrato es la misma que tomamos hoy.

Propuestos para la casa I

- 1 Dígitos.** Escriba `convertir( $n$ )`, que recibe un entero no negativo y devuelve la lista de sus dígitos de izquierda a derecha: `convertir(305) =  $\langle 3, 0, 5 \rangle$` . Hágalo de dos maneras, con insertar al frente y con agregar más `revertir`, y diga cuál hace menos llamadas.
- 2 Paréntesis balanceados.** Con una pila, decida si una cadena de `(, ), [ y ]` está bien balanceada: `([ ])( )` sí, `([ ]]` no. Escriba primero, en palabras, qué se apila y qué se desapila. Thareja, capítulo 7.
- 3 Una cola con dos pilas.** Escriba `encolar`, `desencolar` y `frente` usando solamente dos pilas y su contrato. Trace con `$\langle 7, 2, 9 \rangle$` y una `desencolar` en la mitad.
- 4 Contar en una pila.** Escriba `contar( $p, x$ )`, cuántas veces aparece `x` en la pila, dejándola como estaba.

Resumen I

- Los tres TAD guardan una secuencia $\langle e_0, \dots, e_{n-1} \rangle$. El contrato de cada uno dice por dónde se entra, por dónde se sale y qué se puede leer.
- Lista: cualquier posición, con precondiciones sobre p . Pila: solo el tope, LIFO. Cola: entra por el final y sale por el frente, FIFO.
- Las precondiciones de desapilar, tope, desencolar y frente son la misma: $n > 0$. La implementación las vigila con `assert`.
- Mirar sin destruir tiene precio en la pila (una auxiliar y dos pasadas) y no lo tiene en la cola (dar la vuelta).

Resumen II

- Costos: en la pila y la cola toda operación es $O(1)$; en la lista, $O(1)$ por posición y al final, $O(n)$ para insertar o eliminar en medio. Las tres ocupan $\Theta(n)$. La misma función contra el mismo contrato puede ser lineal o cuadrática según qué operación use: revertir con agregar o con insertar al frente.
- `imprimir`, `mezclar`, `revertir`, `eliminarDesdeTope` y `particionar` se escribieron solo con los contratos. Cuando cambiemos la implementación, ninguna se toca.

Lo que sigue

Los contenedores de la STL que traen estos contratos hechos, y los algoritmos e iteradores que trabajan sobre ellos.

Referencias I

-  R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 2, tipos abstractos de datos; capítulo 7, pilas; capítulo 8, colas.
-  N. Kalicharan. *Data Structures in C*. 2008. Capítulo 4, pilas y colas.
-  R. Sebesta. *Concepts of Programming Languages*. Pearson, 2015. Capítulo 11, tipos abstractos de datos.